

cryptography-journal / week02.md📄

krina808 Update week02.md

b25906e · 24 minutes ago🕒 History

PreviewCodeBlame281 lines (118 loc) · 5.89 KB

ptography Journal

1. Complete the Quiz

- I completed the Week 2 Quiz before the tutorial.
- The quiz helped reinforce my understanding of classical ciphers.
- The questions were well-structured and relevant.

2. Caesar Decrypt (Manual)

- Ciphertext: pjhigpaxp
- Key: 15

Decryption Process:

- Each letter is shifted **backward** by 15 positions in the alphabet.
- Example:
 - p → a
 - j → u
 - h → s
- Decrypted Text: "ausculter"

3. Pycipher for Classical Ciphers

- Installed Pycipher using:

```
pip install pycipher
```

Decryption using Pycipher:

```
from pycipher import Caesar

ciphertext = "pjhigpaxp"

decrypted_text = Caesar(15).decipher(ciphertext)

print("Decrypted Text:", decrypted_text)
```

- Decrypted Text: "ausculter"

Encryption to Verify:

```
encrypted_text = Caesar(15).encipher("ausculter")

print("Encrypted Text:", encrypted_text)
```

- Encrypted Text: "pjhigpaxp"

4. Caesar Decrypt Without Key

- Ciphertext: knuprdv

- Approach: Perform a brute-force shift from 1 to 25.

Correct Decryption:

- After trying all shifts, the correct plaintext was identified using shift 7: o Plaintext: "decrypt"

5. Implement Caesar Cipher in Python

Python Implementation:

```
def caesar_cipher(text, shift, encrypt=True):

    result = ""

    for char in text:

        if char.isalpha():

            shift_amount = shift if encrypt else -shift

            new_char = chr(((ord(char.lower()) - 97 + shift_amount) % 26) + 97)

            result += new_char.upper() if char.isupper() else new_char

        else:

            result += char

    return result
```

Example

```
plain_text = "hello"

shift_value = 3

cipher_text = caesar_cipher(plain_text, shift_value)

print("Encrypted:", cipher_text)

print("Decrypted:", caesar_cipher(cipher_text, shift_value, encrypt=False))
```

Results:

- Encrypted: "khoor"
- Decrypted: "hello"

6. Implement Brute Force for Caesar Cipher

Brute Force Code:

```
def brute_force_caesar(cipher_text):

    for shift in range(1, 26):

        print(f"Shift {shift}: {caesar_cipher(cipher_text, shift, encrypt=False)}")
```

```
brute_force_caesar("knuprdv")
```

- Correct Decryption: "decrypt" (Shift 7)

7. Playfair Decrypt (Manual)

- Ciphertext: rpopizuliz
- Keyword: "program"

Decryption Steps:

- Constructed a 5x5 Playfair Matrix using "program" and the alphabet.
- Applied Playfair cipher decryption rules.
- Decrypted Text: "operation"

8. Rail Fence Decrypt (Manual)

- Ciphertext: irtngapconentehooisnapiaintecledits • Key: 3 Decryption Steps:

- Arranged the text in a zigzag pattern across 3 rails.
- Read the letters sequentially.
- Decrypted Text: "encryptionisachallengingconcept"

9. Rows/Columns Decrypt

- Ciphertext: "cieaexshxettrbxass" • Key: 164325 Decryption Steps:

- Arranged the ciphertext into a 6x3 matrix.
- Applied the row-column transposition using the key.
- Decrypted Text: "thisisatestsuccess"

10. One Time Pad Brute Force

- Hexadecimal One Time Pad
- Decrypting rate: 1010 messages/sec
- Message Length: 300 characters

Worst-Case Time Calculation:

- Total possibilities: 16300
- Time required: 16300/1010
- Result: Infeasible. The One Time Pad remains secure.

11. Optional Task - Rows/Columns and Caesar Decrypt

- Ciphertext: "lhszlplyueshadletip"

https://github.com/krina808/cryptography-journal/blob/main/week02.md

12

- Applied a two-layer encryption:

- Rows/Columns Transposition using a 5-digit key.

- Caesar Cipher using a shift.

Decryption Process:

1. Frequency Analysis: Assumed 'e' as the most frequent letter.
2. Reverse Caesar Shifts: Checked all shifts for readable text.
3. Decrypted Text: "thesolutionwascomplex"

Illustrating Attacks on Systems:

In this case, the brute force attack on the Caesar cypher succeeded because it takes place when the cipher has a small keyspace (1 – 25) and any shift is tried, thus decrypting the message. However, with stronger ciphers such as Playfair or rail fence this is less effective as their keyspace is larger and their encryption processes more complex and harder to automate to execute effectively.

Difficulties in Understanding Security Systems:

Particularly I had trouble understanding how a Caesar cipher’s simplicity can be both its strength and weakness. It is easy to implement and easy to understand, but it is as vulnerable to brute force as anything can be. Not only that, the Playfair cipher uses a 5x5 matrix and the Rail Fence cipher uses a zigzag pattern that made understanding how they were decoded more tricky than the simpler Caesar cipher.

Links to Security Systems:

Link: <https://owasp.org/>

Summary: The free and community driven resources to understand and mitigate security risks in web application are provided by OWASP. It contains guides, tools and frameworks intended for protecting your organisations from common vulnerabilities such as SQL injection, cross site scripting (XSS) and broken authentication. The OWASP Top 10, the most recognized list of the most critical web application security risks is a must have source for developers and security professionals.

Insights and Reflections

1. **Understanding Classical Ciphers:**
 - Through the Caesar, Playfair and Rail Fence ciphers being worked manually, a good understanding was formed of how these early encryption methods function.
2. **Brute Force Vulnerability:**
 - The same caveats about Caesar ciphers having a small key space also made them easy to brute force for these tasks. The brute force attack implementation showed what an automated tool can fast decrypt a simple cipher.
3. **Playfair Cipher Challenges:**
 - Ciphering with the Playfair was another interesting use of matrices and thus also another cipher where deciphering was a painstaking affair.
4. **Practical Use of Pycipher:**
 - Pycipher was a handy exercise in seeing how embedded software tools can take care of said encryption and decryption.